



CYBER-ATTACK DETECTION SYSTEM FOR INTERNET OF HEALTH THINGS (IOHT) DEVICES USING FEDERATED LEARNING

BY

Jenin Mansour (1200540)

Sondos Farrah (1200905)

Alaa Saleem (1200001)

SUPERVISED BY

Dr. Ahmed Shawahna

A Graduation Project Report Presented to the
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
FACULTY OF ENGINEERING AND TECHNOLOGY

In Partial Fulfillment of the Requirements for the Degree of

BACHELOR OF SCIENCE

IN

COMPUTER ENGINEERING

BIRZEIT UNIVERSITY

Birzeit, West Bank, Palestine

AUGUST 2024

To our loved ones,

We dedicate this graduation project to our families, whose constant support and guidance have made this achievement possible.

To our mothers, whose love and prayers give us the strength to keep moving forward.

To Dr. Ahmed Shawahna, our supervisor, for his invaluable guidance and support throughout this journey.

To the souls of our martyrs and the brave prisoners who have resisted injustice.

To our people in Gaza, who continue to fight for their rights despite the destruction they face.

To the young people who have been denied an education due to the ongoing war and genocide.

And to every Palestinian, whether at home or in exile, whose resilience and determination inspire us to keep striving for freedom with unshakable resolve.

Thank you for supporting, inspiring, and motivating us along the way.

Without your love and sacrifice, we would not have been able to complete this work.

Table of Contents

List of Tables	IV
List of Figures	V
List of Abbreviations	VI
Abstract (English)	VIII
Abstract (Arabic)	IX
1 Introduction	1
1.1 Motivation	3
1.2 Problem Statement	4
1.3 Methodology	5
1.4 Contribution	6
1.5 Report Outline	6
2 Background	8
2.1 ECU-IoHT dataset	8
2.2 Cyber-Attack Detection	9
2.3 Homomorphic Encryption (HE)-Based Approach	10
3 Related Work	11
3.1 General Training	11
3.2 Federated Learning (FL)	13
3.3 Homomorphic Learning (HL)	15
4 Proposed Work	18
4.1 Federated Learning (FL) Steps	18

5	Evaluation	26
5.1	Evaluation metrics	26
5.2	Deep Neural Network (DNN)-FL Model Results	28
5.2.1	Summary of Table Results	28
6	Conclusion and Future Work	30
6.1	Conclusion	30
6.2	Future work	31
6.2.1	Future Plan	32
	Bibliography	33

List of Tables

3.1	Design and performance summary of IDS	16
3.2	Key feature comparison: proposed model vs. existing solutions	17
5.1	DNN-FL model results	28
6.1	Future plan	32

List of Figures

1.1	Architecture of our DNN-FL	5
2.1	Type of Attack Distribution	9
2.2	Schematic diagram of HE	10
3.1	Workflow of LW-PWECC approach	12
3.2	Framework architecture and its components	14
3.3	DNN-FL Model architecture	14
3.4	Different federated learning schemes with various types of encryption	15
4.1	Sigmoid Activation Function	19
4.2	Send Global Model to Clients	20
4.3	Batch Size	21
4.4	Epoch	22
4.5	One Sample-Forward Propagation	23
4.6	Send Gradients to the Server	24
4.7	Federated Learning	25
5.1	confusion matrix	26

List of Abbreviations

AI	Artificial Intelligence
ANN	Artificial Neural Networks
ARP	Address Resolution Protocol
ASB-IB	Agnostic Spiking Binarized Neural Network
BLSTM	Bidirectional Long Short-Term Memory
DDoS	Distributed Denial of Service
DL	Deep Learning
DNN	Deep Neural Network
DNS	Domain Name System
DoS	Denial of Service
DT	Decision Tree
FAR	False Alarm Rate
FL	Federated Learning
GRU	Gated Recurrent Unit
HE	Homomorphic Encryption
ICMP	Internet Control Message Protocol
IDS	Intrusion Detection Systems
IID	Independent and Identically Distributed
IoHT	Internet of Health Things
IoMT	Internet of Medical Things
IoT	Internet of Things
LLM	Large Language Model

LW-PWECC Lightweight Privacy and Security Framework using ECC

ML Machine Learning

MLP Multi-Layer Perceptrons

RAG Retrieval-Augmented Generation

RFE Recursive Feature Elimination

SMOTE Synthetic Minority Over-sampling Technique

TCP Transmission Control Protocol

TLS Transport Layer Security

Abstract

With the development and increasing reliance on Internet of Health Things (IoHT) devices in healthcare, these devices have become essential in collecting and analyzing vital health data to support medical decision-making. However, the growing dependence on these devices has introduced significant security risks, exposing sensitive patient data to potential cyber-attacks that threaten privacy and security. This project aims to develop a cyber-attack detection system for IoHT devices using a Deep Neural Network with Federated Learning (DNN-FL) model. Federated Learning provides an effective solution to enhance privacy by processing data locally on the devices without the need for sharing it, helping maintain high accuracy in attack detection.

Homomorphic encryption and cloud computing will be integrated. Homomorphic encryption will allow computations to be performed on encrypted data without decryption, providing an additional layer of security in sensitive environments like healthcare. Cloud computing will enable distributed and efficient data processing, enhancing the system's ability to handle attacks in complex environments while preserving privacy. The system will analyze network traffic to detect various cyber-attacks such as spoofing, data manipulation, ARP spoofing, Denial of Service (DoS) attacks, NMAP port scanning, and Smurf attacks. The system is designed to be applicable in diverse healthcare environments, from hospitals to personal patient devices, ensuring a secure and sustainable environment for connected medical systems. This project provides valuable contributions to both academic and industrial fields in the cybersecurity of smart healthcare.

المستخلص

مع التطور السريع وزيادة الاعتماد على أجهزة الإنترنت للأشياء الصحية في مجال الرعاية الصحية، أصبحت هذه الأجهزة تلعب دورًا أساسيًا في جمع وتحليل البيانات الصحية الحيوية واتخاذ القرارات الطبية. ومع ذلك، أدى الاعتماد المتزايد على هذه الأجهزة إلى ظهور مخاطر أمنية كبيرة، مما يعرض البيانات الحساسة للمرضى لهجمات سبيلانية تهدد الخصوصية والأمان. يهدف هذا المشروع إلى تطوير نظام للكشف عن الهجمات السبيلانية في أجهزة الإنترنت للأشياء الصحية باستخدام نموذج الشبكة العصبية العميقة مع التعلم الفيدرالي. يوفر التعلم الفيدرالي حلاً فعالاً لتعزيز الخصوصية، حيث تتم معالجة البيانات محلياً على الأجهزة دون الحاجة لمشاركتها، مما يساهم في الحفاظ على دقة الكشف عن الهجمات.

لاحقاً، سيتم دمج تقنيات التشفير الهومومورفي والحوسبة السحابية. سيسمح التشفير الهومومورفي بإجراء العمليات الحسابية على البيانات المشفرة دون فك تشفيرها، مما يوفر طبقة أمان إضافية في بيئة حساسة مثل الرعاية الصحية. كما ستتيح الحوسبة السحابية معالجة البيانات بشكل موزع وفعال، مما يعزز قدرة النظام على التعامل مع الهجمات في بيئات معقدة دون التأثير على الخصوصية. يقوم النظام بتحليل حركة مرور الشبكة للكشف عن مجموعة من الهجمات السبيلانية مثل التزييف، تعديل البيانات، هجمات تزييف بروتوكول، هجمات رفض الخدمة، مسح المنافذ، وهجمات. تم تصميم النظام ليكون قابلاً للتطبيق في بيئات متنوعة ضمن مجال الرعاية الصحية، بدءاً من المستشفيات وصولاً إلى الأجهزة الشخصية للمريض، مما يضمن بيئة آمنة ومستدامة للأنظمة الطبية المتصلة. يمثل هذا المشروع إضافة قيمة للمجال الأكاديمي والصناعي في مجال الأمن السبيلاني للرعاية الصحية الذكية.

Chapter 1

Introduction

With the proliferation of Internet of Things (IoT) technologies, which include a wide range of devices that enable the connection between sensors and electrical devices over the Internet, IoT has become an integral part of modern life, including medical services, smart home devices, and global agricultural industries. The interconnected network facilitates interactions between the three main types of relationships human-to-human, machine-to-machine, and human-to-machine, revolutionizing how devices and systems interact in various domains [1]. According to Statista, by 2025, there will be approximately 75 billion IoT devices worldwide, with 30.3% of these devices being devices of the Internet of Healthcare Things Internet of Health Things (IoHT) [2]. In 2024, global expenditures on IoT technology reached 260 billion US dollars, underscoring its economic importance [3].

IoT is widely implemented in a variety of healthcare domains. Smart health care is one of the most promising applications of the IoT. The IoHT is essentially an IoT-based solution specifically designed for health care, combining sensors and healthcare devices to enhance patient care through remote monitoring and real-time data collection. These devices, such as smartwatches with sensors like accelerometers and pulse oximeters, collect highly sensitive health data, for which privacy and security are crucial. The insecure design, implementation, and configuration of IoHT devices exposes them to a variety of cyber-attacks and misuse. These issues make them easy targets and extremely vulnerable to cyber-attacks, such as ransomware, Distributed Denial of Service (DDoS) attacks, botnets, network sniffing, and theft of medical records [4].

To protect IoT networks and nodes, Intrusion Detection Systems (IDSs) are used to track traffic, detect suspicious activities, and mitigate the negative effects of cyber attacks [4]. IDSs is a system that includes various tools and methodologies that examine the system's behavior to identify and prevent attacks or unauthorized access [5]. An intruder detection mechanism, which may exist as a physical device or a digital algorithm, achieves the goal of continuously auditing the interconnected traffic flow. Its primary goal is to identify any potentially malicious packet within the network. Once a malicious packet is detected, the system immediately notifies the user or a designated action unit, which is responsible for taking the appropriate measures to prevent the malicious packet from entering the network [5]. However, IDSs may not be effective in identifying new and unexpected threats. Furthermore, the effectiveness of existing IDSs is questioned as the number of IoT devices increases. This is due to the fact that IDSs searches for anomalies by

monitoring and comparing network data with previously learned patterns or by detecting intrusions based on the relationship between previously learned signatures[4].

IDSs are classified into two types: signature-based and behavior analysis-based. Signature-based IDS detects attacks using known patterns and predetermined signatures or rules, such as specific strings in the network packet or IP addresses associated with attacks. However, while signature-based IDSs are effective against known threats, they struggle to detect complex or zero-day attacks. In addition, they are impractical for IoT devices because of the limited storage and processing capabilities. On the other hand, behavior analysis-based IDSs use Artificial Intelligence (AI) to detect variations from normal patterns, addressing some limitations of signature-based systems [6].

The development of Machine Learning (ML) has significantly impacted the field of cyber security. By integrating ML and Deep Learning (DL) into IDS, it becomes possible to address the growing challenges in cybersecurity. These technologies enable IDS to detect sophisticated threats and adapt to the ever-changing cyber security landscape, making them more suitable for dynamic and resource-constrained IoT environments. Deep learning, in particular, has shown great promise because it can automatically extract valuable features from datasets without human intervention. Techniques such as DNNs have been widely applied in various fields, including network security, demonstrating exceptional performance in intrusion detection [1, 7].

ML is the core technology today for anomaly detection in IoHT devices, but training robust and accurate DL models requires large and diverse data. Health data is highly sensitive, subject to regulations and cannot easily be shared because of regulatory and legal challenges related to ethics, privacy, and data protection. In traditional ML, data from all parties is typically collected and trained centrally. This centralized learning approach poses risks, as once data is collected, users lose control over it, and there is uncertainty about where and how it will be used. This lack of control over sensitive data shows privacy concerns, particularly in the medical domain, such as regulations like the General Data Protection Regulation (GDPR) that prevent hospitals and organizations from sharing sensitive medical data across borders or institutions [4]. In addition to the safety risks, the cyberattack datasets are not available to the general public. To address these issues, we use the publicly available ECU-IoHT [8] dataset, which is unique in this field and represents a variety of cyberattacks.

To address these concerns, research on privacy protection in ML has primarily focused on two main approaches: FL and HE, both of which are being explored as potential solutions.

FL is a privacy-aware and decentralized machine learning paradigm to train a model on private distributed datasets by sending the model to the data instead of uploading the data to a centralized server. Federated learning minimizes the privacy risks associated with traditional centralized machine learning techniques by relying on two core concepts: local computation and model transmission. It involves multiple clients, such as IoT devices to train the model using private datasets dispersed across different devices under the coordination of a central server. The key aspect is that the training data remains decentralized and distributed, so the original data of the client are stored locally and cannot be exchanged or exposed. With the use of federated learning, each device can protect their data privacy by using local data for local training, then uploads only the model gradients to the server for aggregation, and finally the server sends the updated model to the clients.[4, 6, 9]

Despite the advantages of FL, it still faces some challenges in both technological development and deployment, such as the risk of direct data leakage through privacy attacks such as reverse engineering of the trained model. In addition, this approach requires large network bandwidth to send and exchange updated models between clients and the central server. It also requires expensive computational resources on the client side to train the model, which can be a hindrance for devices with limited capabilities[9]. However, to mitigate these challenges, we propose the integration of Homomorphic Encryption (HE) technique with Federated Learning.

Homomorphic Encryption HE is a cryptographic technique that allows computations to be performed on encrypted data without requiring decryption. This property ensures that sensitive data remains secure and private during processing. When integrated with Federated Learning FL, HE can effectively address privacy concerns by encrypting model updates before they are transmitted to the central server. This approach mitigates the risk of reverse engineering attacks and direct data leakage, as the central server never has access to plaintext data or model parameters.

Moreover, HE enhances security in cloud environments where FL operations often occur. By performing encrypted computations on the cloud, HE eliminates the need to expose sensitive information to cloud providers, reducing trust dependencies. This is particularly beneficial for IoHT devices, as it ensures compliance with stringent privacy regulations in the healthcare sector.

In addition to privacy preservation, HE can reduce computational demands on resource-constrained IoHT devices. Since the encryption and secure computation processes are offloaded to the cloud, the computational load on devices is minimized, making FL more practical for devices with limited capabilities. Although HE introduces some computational overhead in the cloud, recent advancements in HE schemes have made them increasingly efficient and scalable, making them suitable for real-world deployment in FL systems.

1.1 Motivation

Over the years, many incidents have occurred that made the need to protect medical devices very urgent and necessary, as these devices contain sensitive medical information they experienced a large number of cyber attacks such as malware that attempts to disable or damage hospital systems, phishing campaigns that steal sensitive data, and DDoS attacks that disrupt hospitals' ability to provide healthcare to patients. These attacks can disrupt hospital operations, endanger patients, and cause financial losses and privacy violations.

The most prominent of these incidents is what happened recently in February 2023, when the Lehigh Valley Health Network (LVHN) in Pennsylvania was attacked by a ransomware attack from the BlackCat group. This attack harmed the personal and medical data of many patients, this data included email addresses, banking information, and patient radiological images. Black Cat demanded a ransom to prevent the publication of the stolen data, but the Lehigh network refused to pay it, which led to them publishing naked photos of patients on the dark web [10, 11].

Another notable incident was the 2017 WannaCry ransomware, which spread to over 150 countries and thousands of computers. This malware encrypts victims' data and prevents

access to it, then demands a ransom to retrieve it. The attack on the National Health Service (NHS) systems in the United Kingdom had the biggest effect, delaying patient care for four days and disrupting with operations in over 80 hospitals. Additionally, in 2020, the University Hospital of Düsseldorf in Germany was the target of a ransomware attack that disrupted the system and prevented access to data, forcing the hospital to turn away emergency patients. A patient who had to travel approximately thirty kilometers to another medical facility during this time sadly passed away as a result of the treatment delay [12].

The most recent incident involved Ascension Healthcare, a nonprofit healthcare organization in the United States, which was targeted by a ransomware attack in May 2024, causing operational disruptions at 120 hospitals. Critical diagnostic services like computed tomography (CT) scans and magnetic resonance imaging (MRI) were disrupted while thousands of systems were encrypted, making electronic health records inaccessible. Also medical staff were forced to work with paper records and manually transfer images to surgeons during operations, which delayed care, increased patient risks, and increased doctors' workload. It took 37 days to restore operations, and during that time, paper records increased significantly. Financially, the company spent around \$130 million responding to the attack and lost \$0.9 billion in operating revenue [13].

The above incidents highlight the serious threat that cyberattacks pose to the healthcare industry, especially with the increase of reliance on the IoHT Devices, despite the fact that they greatly enhance the effectiveness and quality of healthcare, they become an easy target for attacks if cybersecurity is not strong. Therefore, protecting and securing these devices is essential to ensure the continuous and safe delivery of healthcare services.

1.2 Problem Statement

With the increasing use of Internet of Health IoHT devices in the medical sector, these devices have become essential to improving patient care and providing advanced healthcare services. However, their connectivity to the Internet makes them increasingly vulnerable to cyber attacks, putting sensitive patient data and safety at risk, as mentioned in Section 1.1. Healthcare devices often have limited resources, such as power and processing, making it difficult to use traditional protection techniques that may require significant resources such as firewalls that control network data movement, and intrusion detection systems IDS that monitor suspicious activities. In addition, encryption is used to secure data during transmission or storage, but it consumes high processing power unsuitable for resource-limited IoHT devices. This highlights the need for innovative, performance- and security-efficient solutions to address ongoing threats.

The biggest challenge is to protect IoHT devices while ensuring patient data privacy, especially given the increasing reliance on data sharing between devices and centralized servers. Traditional solutions based on centralized data collection can lead to privacy risks. Federated learning is a promising approach that allows for developing cyberattack detection models without the need to share data between devices. However, implementing this technology on IoHT devices requires addressing several challenges, including hardware limitations, model efficiency, and dealing with advanced threats.,

1.3 Methodology

This subsection will go through the approach that we used to implement our project. We will present our architecture, outline the model we chose, give a quick overview of the tools and some important libraries we used, and go over the training and evaluation processes.

Figure 1.1 shows the architecture of our project. The DNN model is initialized and distributed to the client devices (IoHT devices) by the server at the start of the operation. Every client computes gradients locally and trains a local copy of the DNN model using its own private data. After that, these gradients are returned to the server, protecting the privacy of the data and using the least amount of traffic. The central server aggregates the received gradients to update the global model to get better performance and then updated copies of the global model are then sent back to the clients to repeat the process to achieve more accurate results in detecting attacks.

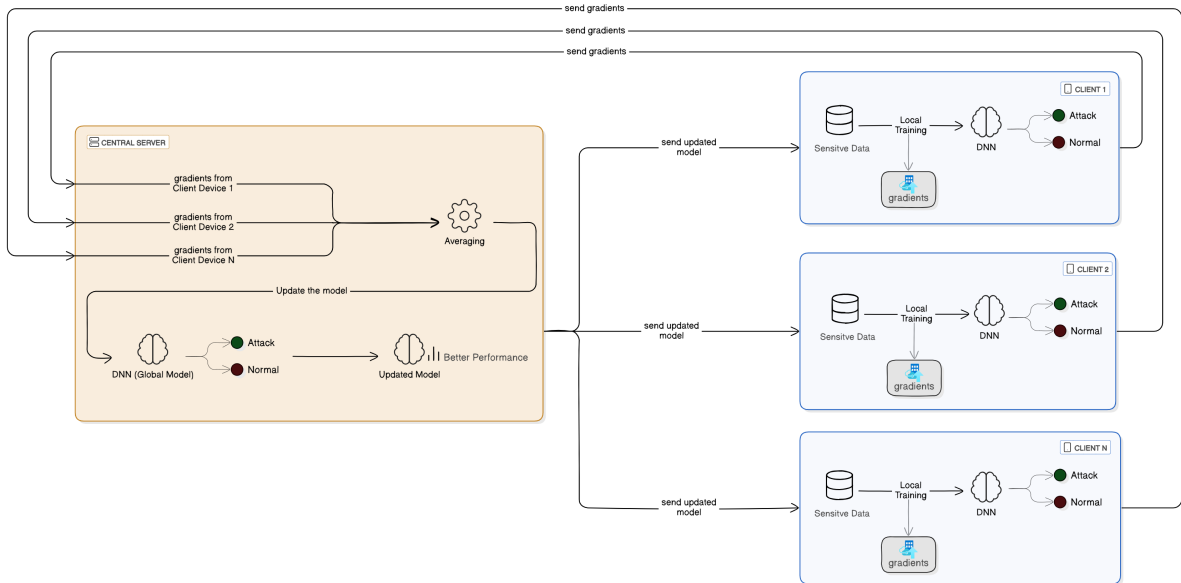


Figure 1.1: Architecture of our DNN-FL

We simulated FL with the ECU-IoHT datasets [8] using TensorFlow’s FL tools. We made the assumption that the data distributions of clients were homogeneous, so the dataset was divided across different numbers of clients, including 5, 10, and 15 clients.

The DNN model that we used has the following architecture:

- **Input Layer:** 6 features
- **Hidden Layers:**
 - Dense layer with 64 units and ReLU activation
 - Dense layer with 64 units and ReLU activation
 - Dense layer with 32 units and ReLU activation
- **Output Layer:** Dense layer with 1 unit and sigmoid activation for binary classification

We used a batch size of 1024 and 100 epochs to train the model. The Adam optimizer was applied to the client and server at different learning rates:

- **Learning rate for clients:** 0.01
- **Learning rate for server:** 0.05

In order to improve model performance, the dataset was preprocessed by scaling and encoding it. Training was distributed among the simulated clients, each working with its local data. Metrics like accuracy, precision, recall, and loss were used to assess the model's performance, indicating how well it could identify different attacks on IoHT devices.

1.4 Contribution

This work presents a method for detecting cyberattacks on IoHT using federated learning. By processing data locally, it improves privacy and security. The goal is to develop a model that prioritizes privacy and avoids attacks while addressing device performance limitations.

The main contributions of this project include:

1. **DNN-FL model:** Developing a deep learning model using federated learning (DNN-FL) for detecting cyberattacks in IoHT.
2. **Privacy enhancement:** Presenting a FL model that contributes to improving the accuracy of attack detection while maintaining data confidentiality without the need to exchange data between devices.
3. **Performance evaluation:** Analyzing the performance of the model using metrics such as accuracy, precision, recall, and loss, and measuring the efficiency of training and testing in different scenarios.
4. **High-accuracy performance:** Achieving accuracy up to 99.8% with low loss and accepted train time, confirming the effectiveness of federated learning to detect cyberattacks.
5. **Scalability:** Providing insights into the challenges associated with limited resources by using the cloud to train models instead of training the model directly on client devices. And using homomorphic encryption to enhance security.

1.5 Report Outline

This report is outlined as follows:

- **Chapter 1: Introduction and Motivation:** Provides an overview of concepts including IoT, IoHT, IDS, ML, FL and Homomorphic Encryption. It also includes the motivation, problem statement, the proposed methodology, and the project's contributions.
- **Chapter 2: Background:** Discusses the dataset used in the project and provides an overview of cyber-attack detection.

- **Chapter 3: Related Work:** Reviews relevant literature and conducts comparative analysis to support the project.
- **Chapter 4: Proposed Work:** Details the proposed solution, It covers the architecture of the federated learning model, and how it is used to detect cyber-attacks for IoHT.
- **Chapter 5: Evaluation:** Evaluates the DNN-FL model and describes the scores using the ECU-IoHT dataset [8].
- **Chapter 6: Conclusion and Future Work:** Summarizes findings, and outlines future research directions.

Chapter 2

Background

This section provides an overview of the key concepts and techniques relevant to our project. We begin by introducing the ECU-IoHT dataset [8], the database on which the project is based. We then discuss the principles of cyberattack detection, highlighting its importance in securing Internet of Health Things IoHT devices. Finally, we explore the concept of homomorphic encryption HE, a critical cryptographic technique that ensures data privacy and security while allowing computational operations on encrypted data, which will be used in future work. This background aims to establish an understanding of the tools and methodologies used in our work.

2.1 ECU-IoHT dataset

The ECU-IoHT dataset [8] was developed as part of a study to analyze the security vulnerabilities and potential cyberattacks on IoHT devices. It is the first of its kind in the healthcare domain, as most other available datasets do not cover IoHT-specific security challenges. The dataset is specialized in analyzing internal networks (such as CAN Bus), making it suitable for training a model to detect attacks in closed environments similar to IoHT device infrastructure. It provides unique metrics such as Unique IP Metrics that help analyze and identify potential sources of distributed attacks (such as DDoS), which is essential for protecting connected medical devices. Its structured and simple nature makes it suitable for applying federation in training, as the data can be segmented based on device locations or unique identifiers, enhancing applicability in your project.

The ECU-IoHT dataset [8] was developed in a controlled environment using a standard white hat penetration testing methodology, incorporating various components such as the MySignals healthcare kit a development platform for IoHT applications along with sensors for temperature, blood pressure, and heart rate. The computing setup included Windows 10, Kali Linux (2020.2), a mobile Wi-Fi hotspot, and a Bluetooth adapter, while software tools utilized were Argus for flow monitoring, TShark for packet analysis, and Ettercap for attack simulation. Notably, the sensor data was transmitted wirelessly to the Libelium cloud server, from which it was captured at the router level for further attack simulation and analysis.

The dataset includes essential fields for analyzing network activity. The number serves as an easy reference point, while the time field records when each packet was captured, helping to identify patterns or delays that may indicate an attack. The source field indi-

cates the device that sent the packet, and the destination field identifies the target device. The protocol field identifies the communication protocol, which is critical to identifying attacks, with common examples being Transmission Control Protocol (TCP), Transport Layer Security (TLS), Internet Control Message Protocol (ICMP), Domain Name System (DNS), and Address Resolution Protocol (ARP). The length field measures the packet size in bytes, helping to detect abnormal behavior, and the information field summarizes the contents of the packet, distinguishing between normal and malicious activity. The type field indicates whether the packet represents a normal connection or an attack, which includes different types such as ARP Spoofing, Denial of Service (DoS), Nmap Port Scanning, and Smurf Attacks, which are identified by the type of attack field.

The ECU dataset contains 111,207 samples, with 78.9% of the sample are classified as attack types, while 21.1% represent normal traffic. The figure 2.1 details the counts of normal traffic like DNS, UDP, and HTTP, as well as various attacks like ARP Spoofing, DoS attacks, Nmap Port Scan, and Smurf attacks.

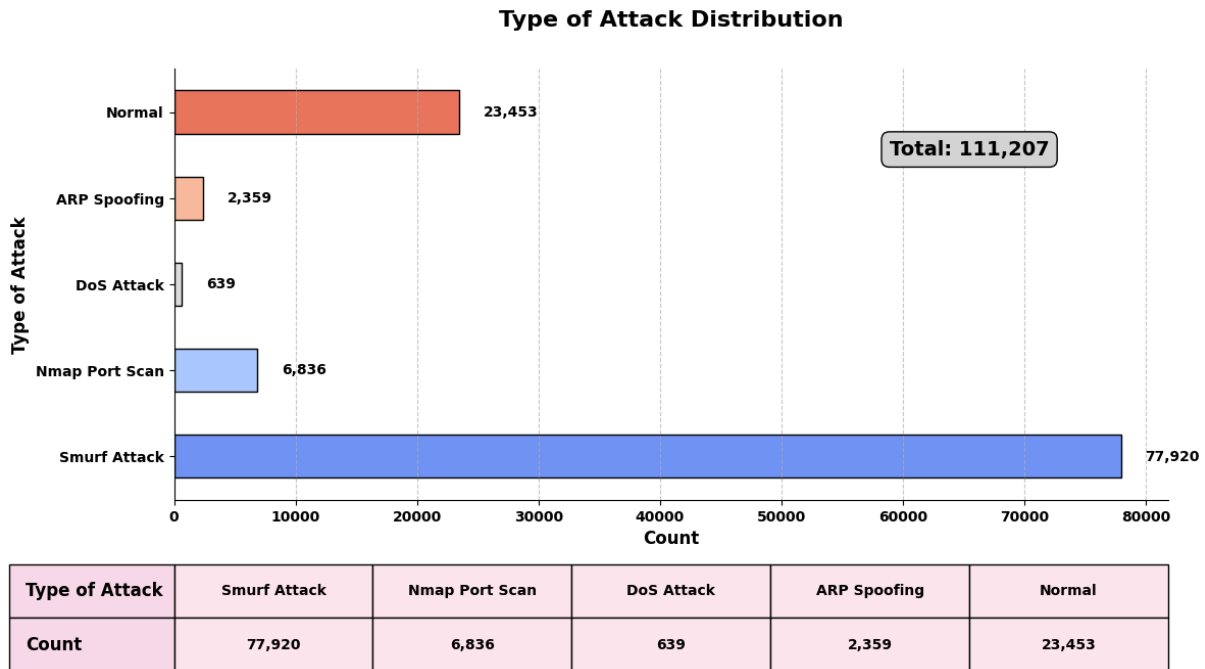


Figure 2.1: Type of Attack Distribution

2.2 Cyber-Attack Detection

Cyber-attack detection is the process of identifying malicious activities and cyber threats within systems or networks before they cause significant damage. This process relies on a variety of techniques, including behavioral analysis, artificial intelligence, and machine learning algorithms, alongside traditional methods such as attack signatures. The primary goal is to detect deviations from normal activity, whether it's a known attack like malware or an unknown threat like zero-day exploits. By integrating advanced tools such as Intrusion Detection Systems (IDS), centralized data analysis platforms (SIEM), and threat intelligence feeds, cyber-attack detection ensures effective identification and response to potential risks.

In the context of IoHT devices, cyber-attack detection plays a vital role in securing highly

sensitive medical data and ensuring the functionality of interconnected healthcare devices. IoHT systems are increasingly targeted by sophisticated cyber threats due to their critical role in patient care and their extensive connectivity. Robust detection mechanisms help proactively identify attacks, mitigate risks, and protect both patient safety and data integrity. By leveraging advanced detection techniques tailored to IoHT environments, healthcare systems can enhance their resilience against emerging cyber threats and maintain trust in smart medical technologies.

2.3 HE-Based Approach

Homomorphic encryption is a cryptographic technique that performs computations directly on encrypted data without decrypting it. This method ensures the confidentiality of data throughout the computation process. Homomorphic encryption schemes can be classified into Partially Homomorphic Encryption (PHE) and Fully Homomorphic Encryption (FHE). PHE supports a limited set of operations (e.g., addition or multiplication) on encrypted data, while FHE allows arbitrary computations[14].

For example, in PHE, the Paillier cryptosystem supports additive homomorphism. If c_1 and c_2 are ciphertexts of plaintexts m_1 and m_2 , encrypted as $c_1 = g^{m_1} \cdot r_1^n \mod n^2$ and $c_2 = g^{m_2} \cdot r_2^n \mod n^2$, the product $c_1 \cdot c_2 \mod n^2$ yields $g^{m_1+m_2} \cdot (r_1 \cdot r_2)^n \mod n^2$, which decrypts to $m_1 + m_2$. On the other hand, FHE, introduced by Gentry in 2009, supports addition and multiplication, enabling any computation to be represented as a series of Boolean circuits. A key feature of FHE is its ability to "bootstrap," where a ciphertext can be refreshed to reduce noise, ensuring the encryption scheme remains functional for complex computations. Despite its promise, FHE faces challenges due to high computational overhead and large ciphertext sizes[14].

Because of the need for accuracy and speed in decision-making, the client's calculation process takes time and is unacceptable, homomorphic encryption is important. Each IoHT device (client) encrypts its data using its public key and then sends the encrypted data to the server. The server collects the encrypted data from all devices. It performs calculations on them directly using homomorphic encryption, without decrypting them, ensuring that sensitive data remains private and inaccessible to the server. The server then uses the model to classify patterns in the data, whether they indicate an attack or normal action. The classification result, which remains encrypted, is sent to the relevant client. Upon receipt of the result, the client decrypts it using its private key to determine whether a cyber attack has been detected or not[15].

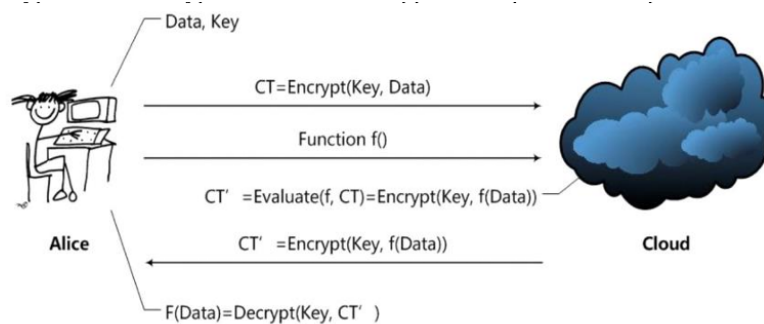


Figure 2.2: Schematic diagram of HE [9]

Chapter 3

Related Work

This section highlights the most important contributions from previous studies related to our project. We have divided previous studies into sections to facilitate understanding and access according to the methodology followed by the studies. We begin with studies that followed general training methodologies and their limitations in privacy-preserving environments. We then review previous studies that followed the Federated Learning FL methodology, a decentralized approach designed to address privacy concerns in distributed systems. This is the approach we followed in our project. Finally, we apply Homomorphic Encryption HE as an emerging technique for secure data processing for use in future work. By reviewing these works, we aim to place our study in the broader context of existing research and identify gaps that our approach seeks to address.

3.1 General Training

Study [1] introduced a hybrid deep learning-based IoHT-IDS to assess how well deep learning techniques detect cyberattacks in IoHT systems. A Artificial Neural Networks (ANN) component that effectively handles complex patterns present in IoHT data, a Bidirectional Long Short-Term Memory (BLSTM) layer that records bidirectional dependencies, and a Gated Recurrent Unit (GRU) layer that is helpful at managing long-term sequential features are all included in the model. The model was tested using ECU-IoHT dataset [8], and achieved performance with accuracy Above 0.99, and F1-Score above 0.997.

In [7], vijayakumar et al. proposed a DNN model for detecting cyber attacks within IoHT systems. The model is trained using the ECU-IoHT dataset[8], consisting of 111,207 samples, and employs a DNN architecture with an input, two dense layers, and an output layer equipped with a softmax activation function for classifying normal and abnormal activities. The accuracy of the suggested proposed was 99.85.%

Ranjith et al.[16] introduce a cryptographic framework, Lightweight Privacy and Security Framework using ECC (LW-PWECC) shown in fig 3.1, for securing data transmission in IoT healthcare systems and a novel attack detection method based on deep learning. Specifically, they use an Agnostic Spiking Binarized Neural Network (Agnostic Spiking Binarized Neural Network (ASB-IB)), optimized with the Billiards Optimization Algorithm, to improve detection accuracy and stability, achieving 99.93 accuracy. The LW-PWECC framework employs elliptic curve cryptography with a puzzle optimization to enhance

data security, minimizing delay and resource use. Compared to other methods, this approach shows significant improvements in training stability, security, and computational efficiency.

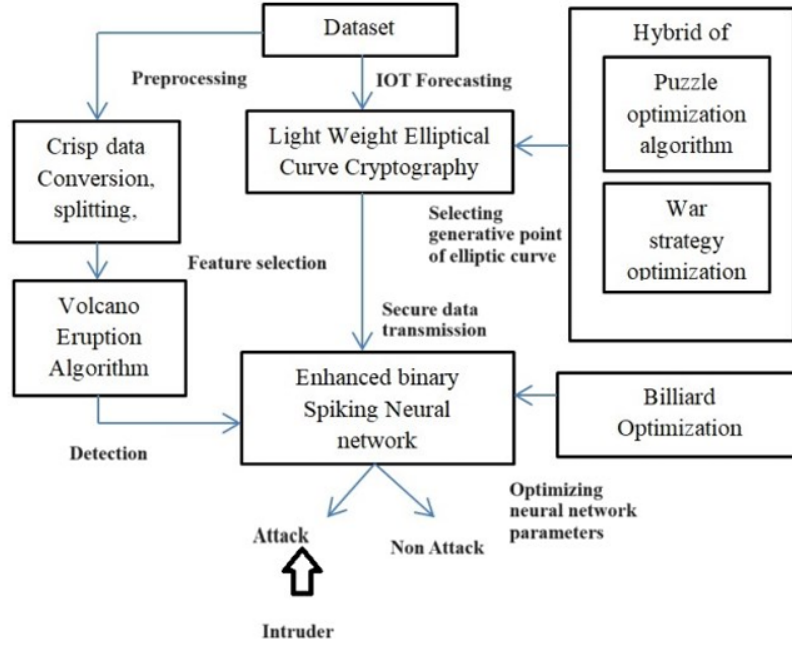


Figure 3.1: Workflow of LW-PWECC approach [16].

Lazrek et al. [17] proposed an advanced IDS for the Internet of Medical Things (IoMT) that uses Recursive Feature Elimination (RFE) associated with ML and Ridge regression with DL models to detect anomalies in real-time datasets. Their model made use of the WUSTL-EHMS dataset, which focuses on network traffic abnormalities in healthcare systems, especially those caused by cyber-attacks on IoMT devices. The suggested RFE-Decision Tree (DT) model exceeded existing approaches, with 99% training accuracy, 97.85% testing accuracy, 96.50% precision, 92.92% AUC, and a False Alarm Rate (FAR) of only 0.03. This model exceeded classic machine learning algorithms and other new models. The article notes that, although excellent accuracy and precision, further improvements in recall can be achieved through ensemble methods such as voting classifiers or stacking.

Umer Zukaib et al. [18] proposed Meta-IDS, a meta learning based intrusion detection system for IoMT networks that uses signature-based and anomaly-based detection to improve security. Their system detects real-time threats, including zero-day attacks, using ML and DL models, hyperparameter optimization and feature engineering techniques such as Synthetic Minority Over-sampling Technique (SMOTE) and mean-shift clustering. It was tested on three datasets-WUSTL-EHMS-2020, IoTID20, and WUSTL-IIOT-2021 and performed well. Meta-IDS also performed well compared by the well-known model E-GraphSAGE in terms of accuracy, precision, recall, F1 score, and time complexity, demonstrating its reliability over a wide range of attack situations.

This study [5] presents an innovative IoT intrusion detection framework using the Three-Way Selected Random Forest (3-WSRF) model, which improves on traditional approaches by employing entropy-based attribute classification and gray wolf optimization for enhanced detection accuracy and adaptability. The model categorizes data into normal, abnormal, and uncertain, addressing challenges of noise, outliers, and imbalanced datasets.

The 3-WSRF method demonstrates robustness, scalability, and efficiency while being seamlessly integrable into existing IoT security architectures. Despite its promising results in improving IoT network security, further validation on diverse datasets and larger infrastructures is recommended for future research.

Phani Lanka et al. [19] suggested a machine-learning-based method for analyzing high-interaction honeypot records and correlating unusual activity with critical infrastructure. The approach uses K-Means clustering to differentiate between automated and human-driven sessions, and Retrieval-Augmented Generation (RAG) for command analysis. Using advanced Large Language Model (LLM)s like GPT-4-turbo and embedding models such as Salesforce/SFR-Embedding-Mistral, the system effectively recognizes and categorizes malicious commands according to the MITRE ATT&CK methodology. The proposed approach performed well, with a silhouette score of 0.98% and an F1 score of 0.91%, indicating that it provides a reliable mechanism for early detection and reaction to attacks aimed at critical systems.

In [12] paper, the authors proposed a security model based on LightGBM and Transformer algorithms to protect healthcare systems from cyberattacks. The proposed system consists of two components: an intrusion detection system to monitor Internet of Medical Things (IoMT) devices, and a malware detection system to secure staff computers. The model was trained on four different datasets to cover a variety of attack types that could potentially target the healthcare sector. Experimental results showed high accuracy, demonstrating the effectiveness of this hybrid approach in providing comprehensive protection across diverse healthcare environments.

3.2 Federated Learning (FL)

The study by Rey et al. [20] introduces a federated learning framework aimed at detecting malware in IoT devices, addressing privacy and efficiency in data sharing. Using the N-BaIoT dataset, their model which shown in fig 3.2 was trained to identify malware on both familiar and unfamiliar IoT devices via federated learning. They experimented with multi-layer perceptron and autoencoder models, comparing federated learning with localized and centralized approaches. The framework demonstrated high accuracy while maintaining privacy, though it faced challenges from adversarial attacks; alternative aggregation methods were tested to improve robustness against such threats.

The study [6] introduces a semi-supervised distributed learning method using FL-based IDS for large-scale environments due to the increasing number of IoT devices. The experiment is performed in a decentralized environment using UMBRELLA nodes powered by Raspberry Pi 3+ and Jetson Nano devices. The dataset is custom-generated via controlled experiments, includes three scenarios: a DoS attack, a privilege escalation attack, and normal operations. Data collection is done by the Linux auditing system (auditd), which records Linux system calls (SYSCALL) that are analyzed using an **autoencoder** neural network to identify abnormal activities, the system then detects anomalies by observing the reconstruction errors in feature vectors. The FL approach is evaluated using three experiments across edge devices with varying pre-training proportions (100%, 90%, and 50%). Results show that fewer clients participating in an FL setup, lead to higher performance. Furthermore, starting with a pre-trained model improves the overall performance.

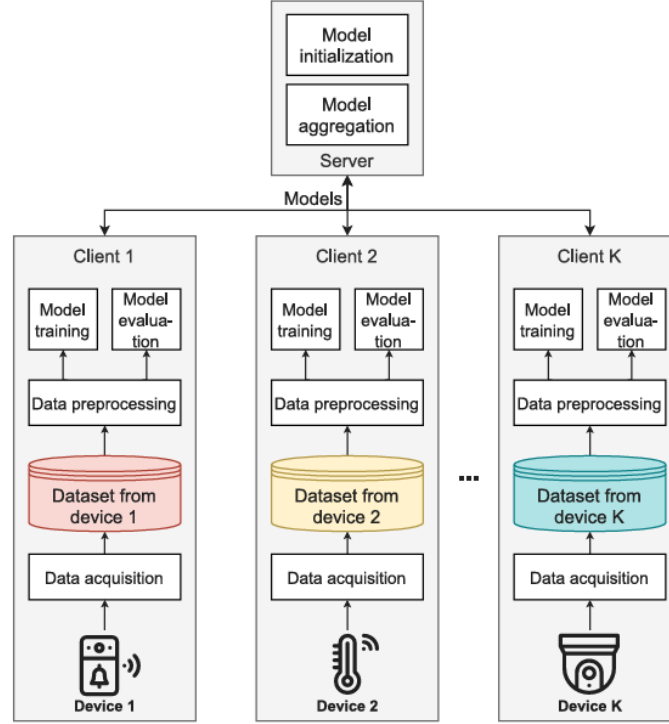


Figure 3.2: Framework architecture and its components [20]

FL is a machine learning framework that effectively detects anomalies in a decentralized environment while protecting data privacy. **Paper** [4] implemented a DNN with FL to detect cyberattacks in IoHT devices. The server distributes a DNN model to clients, who use local data to train on their model. The IoT device sends the model to the central server, which aggregates it using the averaging algorithm, and returns the aggregated global model to IoT devices for the next iteration. The model was trained and tested on two datasets: **wustl-ehms-2020** (with 91.40% accuracy) and **ECU-IoHT** [8] (with 98.47% accuracy).

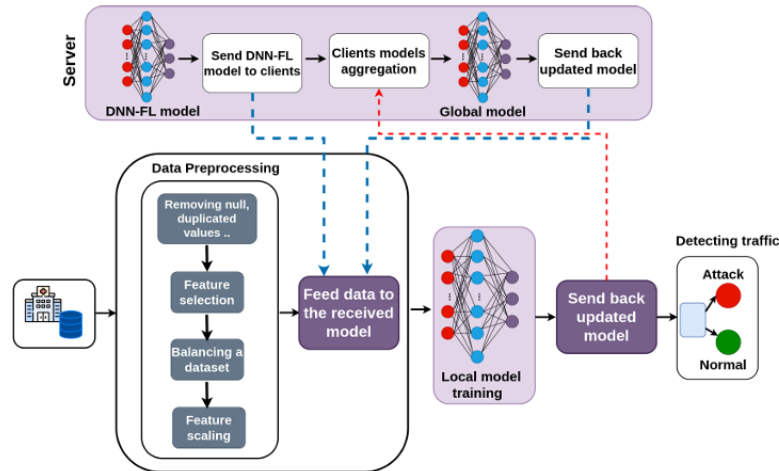


Figure 3.3: DNN-FL Model architecture [4]

3.3 Homomorphic Learning (HL)

FL with Multi-Key HE model is introduced by the study in [15]. The study highlighted privacy issues with FL as shown in Fig.3.4, FL improves privacy by avoiding data sharing during training, but local model updates can still expose confidential information. Sharing local data with a server can endanger privacy protection of the devices, as any participant with access to these updates could potentially see them.

Participants in a model encrypt local model updates using an aggregated public key before sending them to the server. The server computes the global model by aggregating these encrypted updates without decrypting them. Participants must collaborate through decryption shares to decrypt aggregated cipher-text, ensuring no party can access aggregated data or individual updates without authorization. The UP-FALL dataset, which contains sensor data for fall detection in the elderly, was used to assess the model.

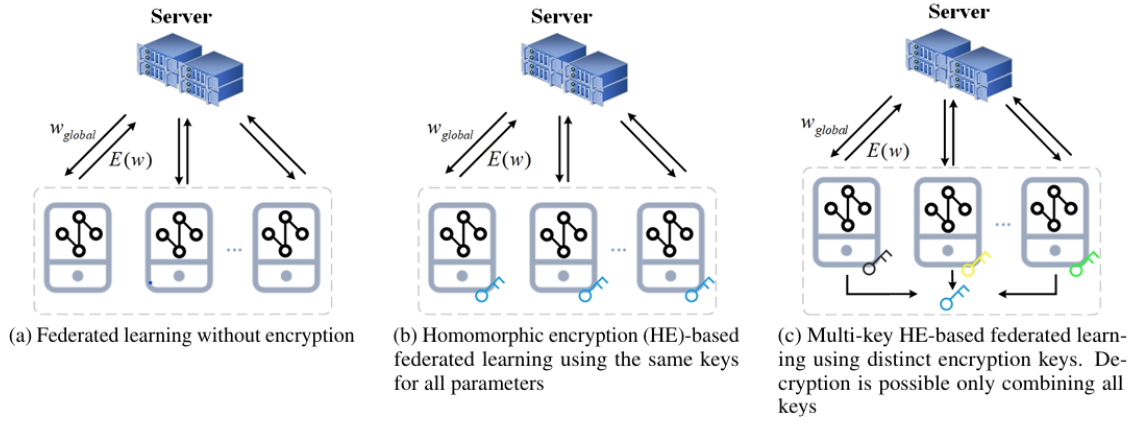


Figure 3.4: Different federated learning schemes with various types of encryption [15]

In **Table 3.1**, we present a summary of previous works, highlighting their datasets, techniques, accuracy, and limitations, providing a comprehensive overview of the methodologies and gaps in the literature. Then we compared the related works in **Table 3.2** based on model type, code availability, IoHT environment, and use of federated learning. After comparison we found that the works that are most closely related to our research are [4], [20], and [18].

Table 3.1: Design and performance summary of IDSs.

Article	Used Datasets	Technique	Accuracy	Limitations
[4]	ECU-IoHT WUSTL-EHMS-2020	DNN-FL	ECU 98.47%, wustl 91.40%	does not detail the dataset distribution or analyze Global Model initialization.
[7]	ECU-IoHT	DNN	99.85%	Centralized data sharing; lack of privacy.
[1]	ECU-IoHT	Hybrid deep learning	99.85%	lack of data privacy
[16]	ECU-IoHT	LW-PWECC	99.93%	Lack of well-defined standards in IoHT architecture by policymakers. Insufficient adoption of cloud infrastructure. Potential for new types of cyberattacks. Need for further research to improve the IoHT ecosystem.
[20]	N-BaIoT	FL (Multi-Layer Perceptrons (MLP), Autoencoder)	95%-98% (MLP) 90%-93% (Autoencoder)	Vulnerable to adversarial model poisoning, requires careful handling of non-Independent and Identically Distributed (IID) data distributions.
[18]	WUSTL-EHMS-2020 IoTID20 WUSTL-IIOT-2021	Meta-learning	99.57%, 99.93% (signature-based) 99.47%, 99.98% (anomaly-based)	Resource-intensive training and difficulty in optimizing for real-time execution and cross-domain applications
[17]	WUSTL-EHMS	RFE/Ridge-ML/DL	99% training accuracy, 97.85% testing accuracy	Needs recall improvement, can be addressed with ensemble methods.
[6]	custom-generated	autoencoder-based-FL	Not clear	Performance relies on data distribution and pre-training, limiting scalability, and using only 2 clients restricts robustness testing in larger networks.
[19]	High-interaction honeypot logs	K-Means, RAG with GPT-4-turbo	Silhouette: 0.98% F1: 0.91%	Limited to application-layer analysis; does not address non-application-layer DoS attacks. Requires multi-honeypot setups and extended timeframes for broader analysis.

Table 3.2: Key feature comparison: proposed model vs. existing solutions.

Reference	Model	Code– availability	IoHT- environment	Federated- Learning- based
[4]	DNN-FL	✓	✓	✓
[6]	autoencoder- based-FL	×	×	✓
[1]	Hybrid DL	×	✓	×
[7]	DNN	×	✓	×
[20]	multi-layer perceptron & autoen- coder based FL	×	×	✓
[12]	LightGBM & Trasnformer	×	✓	×
[5]	3-WSRF	×	×	×
[16]	LW- PWECC & ASB-IB & DL	×	✓	×
[19]	K-Means, RAG with GPT-4- turbo	×	×	×
[17]	RFE/Ridge- ML/DL	×	✓	×
[18]	Meta- learning	×	✓	×
Our work	DNN-FL	✓	✓	✓

Chapter 4

Proposed Work

In this section, we gave a general overview of Federated Learning (FL) and its idea. Following that, we described in detail how to use deep learning specifically the DNN-FL model to apply FL to our Intrusion Detection System (IDS). A detailed explanation of each training step was given, including both the Federated Learning and Deep Neural Network (DNN) aspects. Finally, we presented our suggested system's architecture.

FL can be considered as a kind of ML that was created to protect sensitive data. Traditional ML uses a centralized that is trained on a server after collecting data from many resources. However, data security and privacy issues came out, particularly with sensitive data. This is addressed by Google, it introduced FL in 2016, which changes the paradigm: instead of bringing data to the model, the model is sent to the data. This method protects the privacy of data and has been used extensively in real-world fields like banking and health care, where sensitive and private data is present.

The IDS for the IoHT devices is the main focus of our project. Training a ML model to identify possible threats to IoHT devices is the aim. However, since training the model requires data, and this data is sensitive, we cannot afford to transfer it to a centralized server. As a result, we choose to train the model while protecting the data by using FL.

4.1 FL Steps

Step1: Initiating the Global Model

We assume that our learning model is a DNN model. Here's how the global model is initialized:

1. Input Layer: This is the first layer of the model where the DNN receives the dataset's features. The number of features is equal to the number of neurons in this layer. In this layer and the layers that follow, every neuron calculates its output using the activation function

$$a(w \cdot x + b) \tag{4.1}$$

where x is the neuron's input, w is its weights, and b is its bias. The network is able to learn complex patterns by applying the activation function a to the weighted sum of the inputs.

2. Hidden Layers: The number of hidden layers and neurons in each one is chosen to balance model performance and computing efficiency as these layers process the data

from the input layer. The network can identify complex patterns that basic linear models can miss due to hidden layers.

3. **Output Layer:** this is the last layer which provides the prediction. Right now, the output layer is a single neuron because our scenario is binary classification, i.e., identifying whether or not there is an attack. In this layer, the sigmoid activation function is employed, which is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (4.2)$$

The chance of an attack is indicated by the probability score that this function produces, which ranges from 0 to 1.

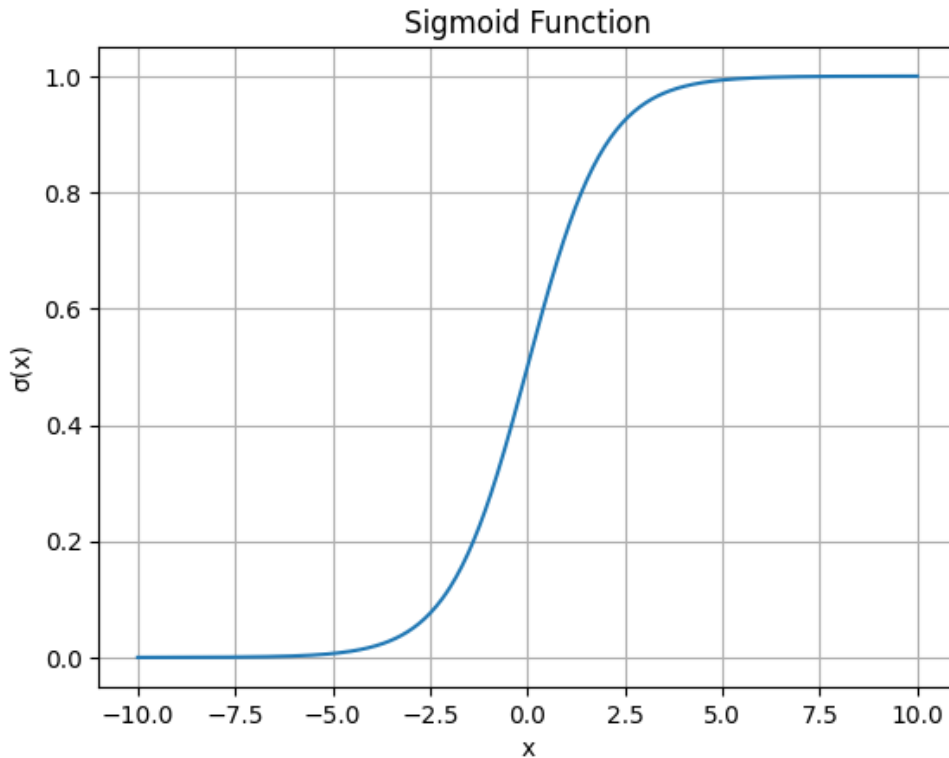


Figure 4.1: Sigmoid Activation Function

Step2: Distributing the Model to Clients

The global model must be distributed to the clients after it has been initialized on the server. In this context, clients are users' edge devices, which in our case are IoHT devices that are used by multiple individuals. We want to keep the personal information on these devices private. But in order to improve the model's performance, we also want to train it on this data. Thus, as shown in figure 4.2, the server sends the initialized global model to every client device. Here, it's important to remember that data never leaves the client device. The model on the client device is known as the local model once it gets there.

Step3: Local Training on Clients

Each client uses its sensitive data to train the local model, (which is a copy of the global model) after receiving it. The training procedure is described in detail in the following steps:

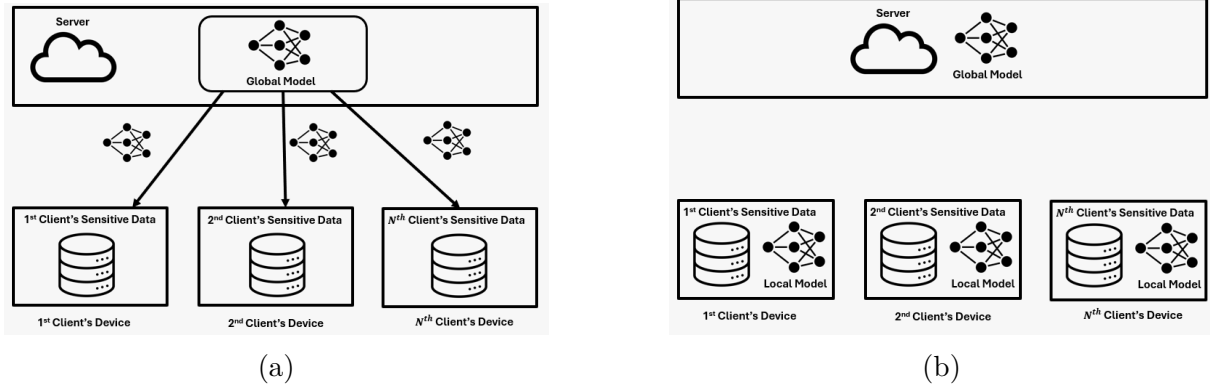


Figure 4.2: Send Global Model to Clients

1. Each client divides their local data into two parts: training and testing data. Generally, 20% of the data is used for testing and 80% for training; the data is randomly shuffled to provide a balanced distribution.

Note: We assume a homogeneous FL environment in our project. It means that the feature set and structure of the data from every client are the same (for example, the quantity and kind of input features) and this is different from Heterogeneous FL.

2. Training Settings (Batch Size and Epochs): Batch Size and Epochs are important parameters that affect how the model learns from the data during the training step. The number of samples took in a single training iteration is referred to as the batch size. For example, if we set the batch size to 1, the model will process each sample individually, which could improve accuracy as the model learns from every sample one by one. However, training will take a lot longer if we have one million samples because each one needs to be processed separately. The batch size can be increased to speed up the procedure. The model will handle the first 100,000 samples together, followed by the following 100,000, and so on, if the batch size is set to 100,000. With this method, the work is completed in 10 iterations rather than 1 million, greatly cutting down on training time. The model's finer adjustments with each sample can produce more accurate results with smaller batch sizes, but there is a trade-off: bigger batch sizes speed up training, but accuracy may be marginally decreased. To balance training speed and accuracy, the appropriate batch size must be determined.

Epochs specify how many times the complete dataset is run through the model. The model will run over the dataset five times if Epochs is set to five. here is a trade-off between size of batch and epochs, The model's performance stability may be enhanced by increasing the batch size, but training may be slowed down if it is set too high. Similarly, while adding more epochs may increase model accuracy, but too many epochs could lead to overfitting, so it's essential to find a balance between Batch Size and Epochs.

3. Forward Propagation

The process starts by feeding the input data into the first layer of the neural network. A linear combination is produced in each hidden layer of the DNN by multiplying the inputs by the weights and then adding the bias. The hidden layer's outputs are obtained by passing these values through an activation function. common activation functions include ReLU (Rectified Linear Unit), Sigmoid, tanh, Linear, etc. The model gains non-linearity from the activation function, which enables it to recognize intricate patterns in

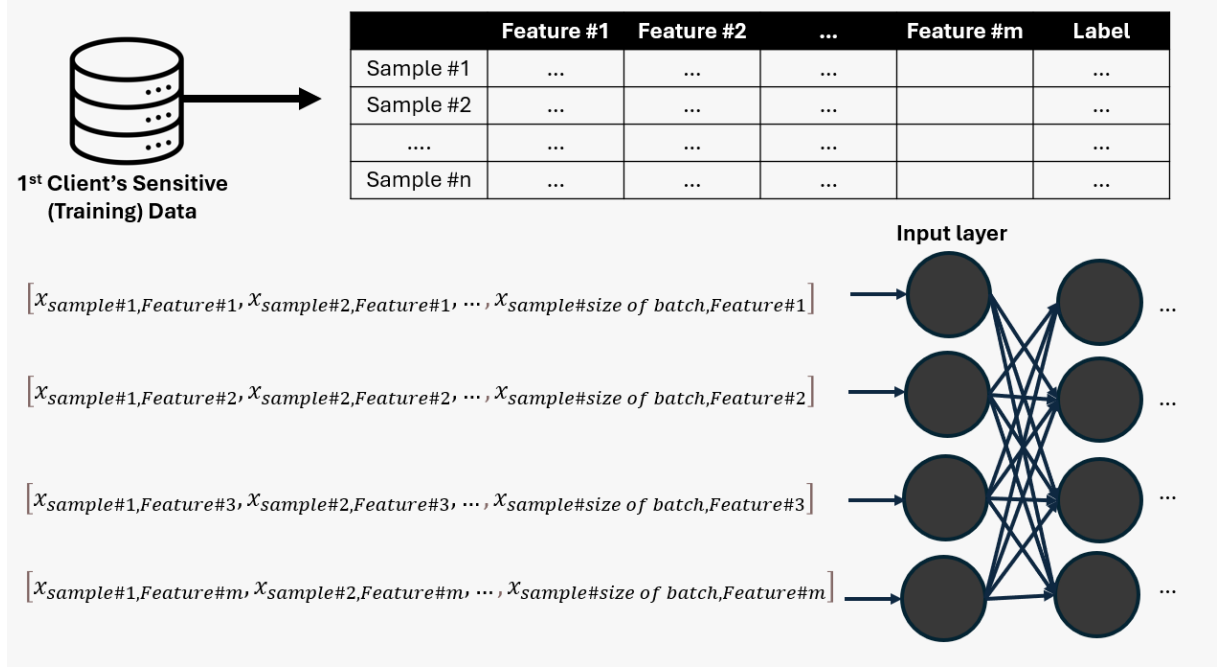


Figure 4.3: Batch Size

the data. The data arrives at the output layer after going via the hidden layers. The output layer uses the Sigmoid activation function because our task is binary classification (i.e., identifying whether an attack is there or not). The chance of an attack is indicated by the sigmoid function's output, which ranges from 0 to 1. An attack is predicted by the model if the output is closer to 1; if it is closer to 0, no attack is identified. The forward propagation method for a single sample with n features is shown in Figure 4.5.

4. Calculating Loss Function

One metric for assessing how well the model predicts the true values (true Labels) is the Loss Function. Cross-Entropy Loss is commonly utilized to measure the difference between the true labels and predicted probability in classification tasks. The true label for each sample is represented as a one-hot encoded vector, with all other indices set to 0 and just the index corresponding to the correct class set to 1. With each class having its own predicted probability, this encoding guarantees that the model produces a probability distribution across all potential classes.

The Cross-Entropy Loss is calculated using the following equation:

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i) \quad (4.3)$$

Where:

- C : The number of classes.
- y_i : The true label for class i .
- $\hat{y}_i$: The predicted probability for class i .

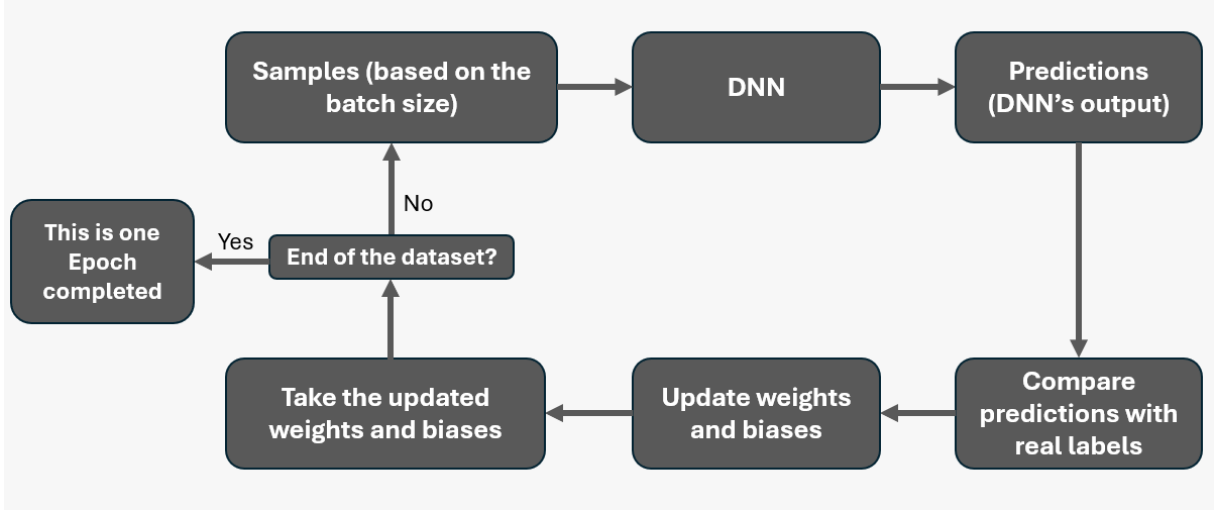


Figure 4.4: Epoch

According to this equation, the Loss decreases as the expected probability $\hat{y}_i$ near the true label y_i . The model's performance is supposed to be enhanced by minimizing the Loss during training.

5. Backward Propagation

In neural network training, backward propagation is an essential procedure, it ensures that the model learns effectively by modifying its weights and biases in accordance with the error determined after the prediction. The following is a brief explanation:

a. Calculating the Gradient: To calculate the contribution of each weight and bias to the error, gradients are computed using the calculated error (loss).

b. The output layer gradient: Gradients for the weights and biases of the final layer are computed by differentiating the loss L in relation to each weight w . A weight w example might be:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w} \quad (4.4)$$

c. Gradient for Hidden Layers: In order to take into consideration all dependencies, gradients are propagated backward across the layers using the chain rule.

Note: Biases are modified according to how much they contribute to the error, exactly as weights:

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b} \quad (4.5)$$

The error is decreased by updating the weights and biases after the gradients have been calculated. The Gradient Descent Algorithm is used to do this. Here are the weight and bias update formulas:

$$w_{t+1} = w_t - \eta \cdot \frac{\partial L}{\partial w} \quad (4.6)$$

Where:

- w_t : Current weight at iteration t .

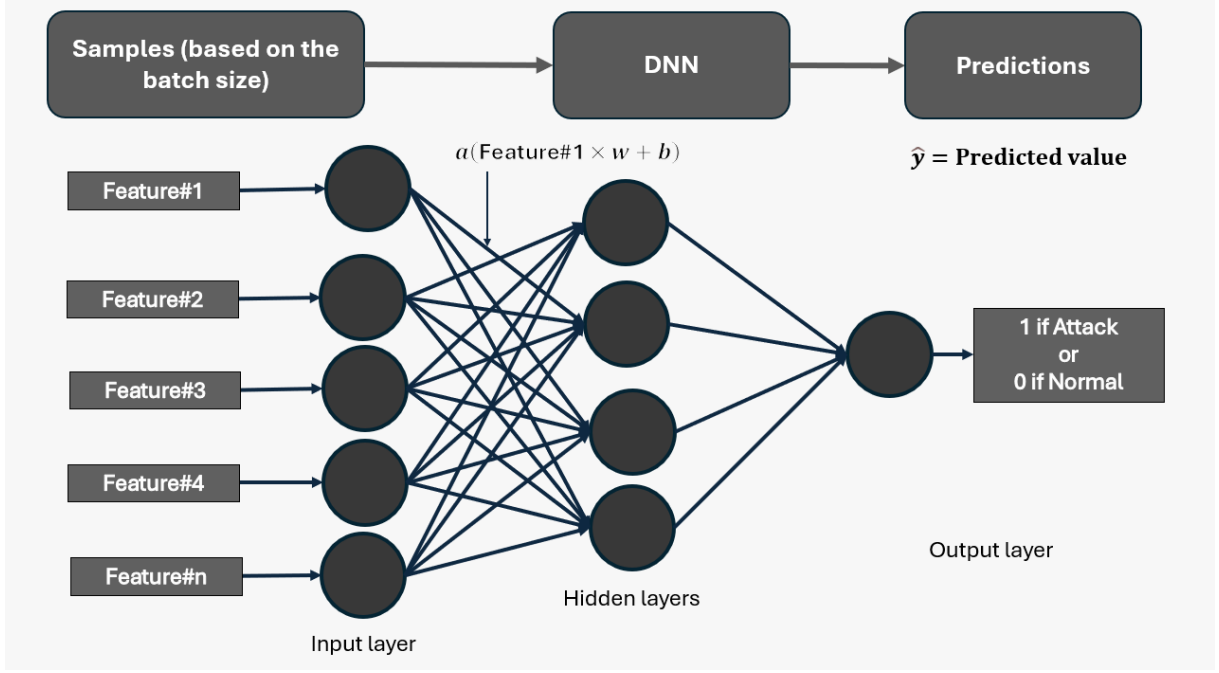


Figure 4.5: One Sample-Forward Propagation

- w_{t+1} : Updated weight.
- η : Learning rate (controls the step size for updates).
- $\frac{\partial L}{\partial w}$: Gradient for the weight.

$$b_{t+1} = b_t - \eta \cdot \frac{\partial L}{\partial b} \quad (4.7)$$

Where:

- b_t : Current bias at iteration t .
- b_{t+1} : Updated bias.
- η : Learning rate (controls the step size for updates).
- $\frac{\partial L}{\partial b}$: Gradient for the bias.

This procedure is repeated again until the entire dataset has been processed (one epoch). Biases (b) and weights (w) are updated during and after each epoch. For the predetermined number of epochs, the process is repeated, beginning with the most recent updates to w and b . The training procedure is finished once all epochs are completed. See figure 4.4.

Step 4: Sending Gradients to the Server

The client sends the calculated gradients to the server instead of sending the raw data or the local model after computing the gradients using its local data. There are a number of benefits to sending gradients rather than raw data or local model parameters. Compared to model parameters, which could inadvertently provide dataset patterns, gradients do not

directly provide information about the original data, providing more privacy protection. Furthermore, gradients are usually smaller, which saves bandwidth and improves system performance, particularly in distributed environments.

By limiting the sharing of any sensitive information between the clients and the server, this guarantees data privacy. The gradients are transmitted as a vector of gradients, where each component denotes the modification needed for every model weight. If the gradient is, for instance

$$\nabla L = [0.01, -0.02, 0.03]$$

These values, which indicate the required modifications for every weight in the model, are transmitted to the server, as shown in figure 4.6.

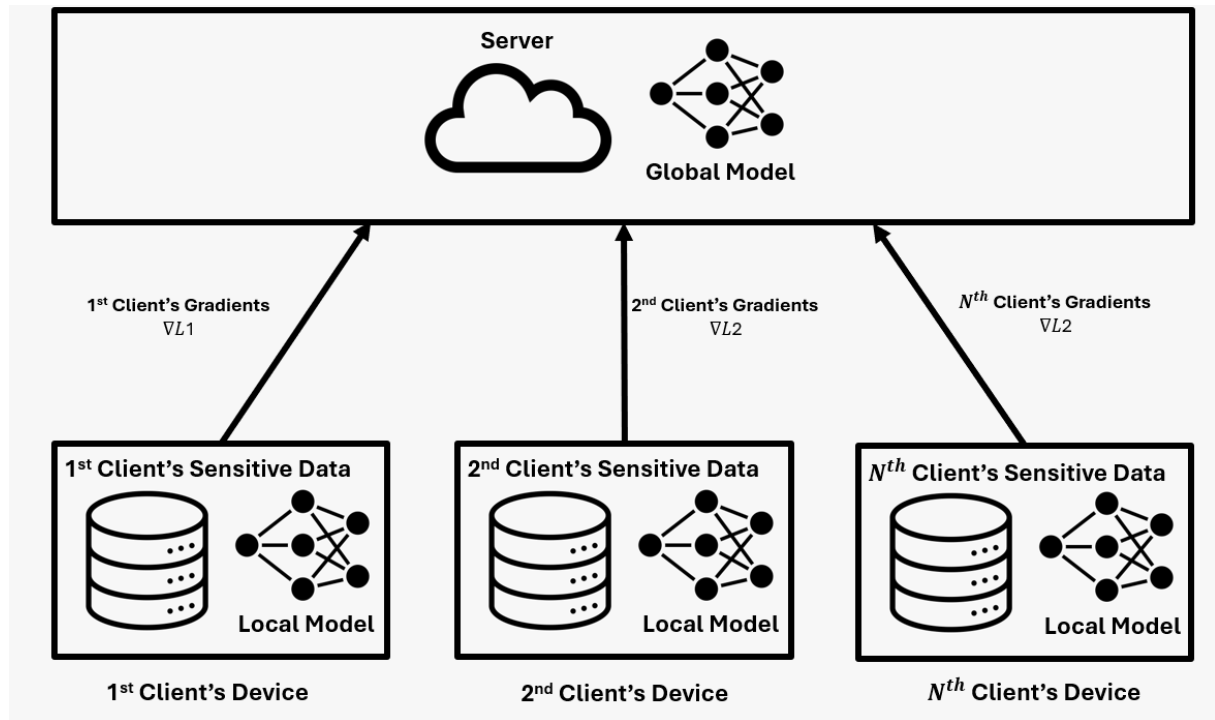


Figure 4.6: Send Gradients to the Server

Step 5: Averaging

The server calculates the average of all the gradients it has received after receiving each one from the clients. This procedure guarantees that the information obtained from the various clients is taken into consideration when the model is updated. The following is the formula for averaging:

$$\Delta \mathbf{L}_{\text{global}} = \frac{1}{N} \sum_{i=1}^N \Delta \mathbf{L}_i \quad (4.8)$$

Where N is the number of clients participating in the training, and $\Delta \mathbf{L}_i$ is the gradient sent by client i .

Step 6: Updating the Global Model

The weights in the global model are updated by the server using the average of the gradients that were recently computed. The formula for updating is:

$$\mathbf{L}_{\text{global}} = \mathbf{L}_{\text{global}} + \eta \cdot \Delta \mathbf{L}_{\text{global}} \quad (4.9)$$

Where $\mathbf{L}_{\text{global}}$ are the current values of parameters of the global model, η is the learning rate, and $\Delta \mathbf{L}_{\text{global}}$ is the average gradient calculated from the received gradients. Without transferring the actual data between clients and the server, this approach allows the global model to be updated by all clients and improved to reflect the diversity of data.

The clients are then provided the updated global model, and the process continues repeatedly, progressively improving the model with each cycle as shown in figure 4.7.

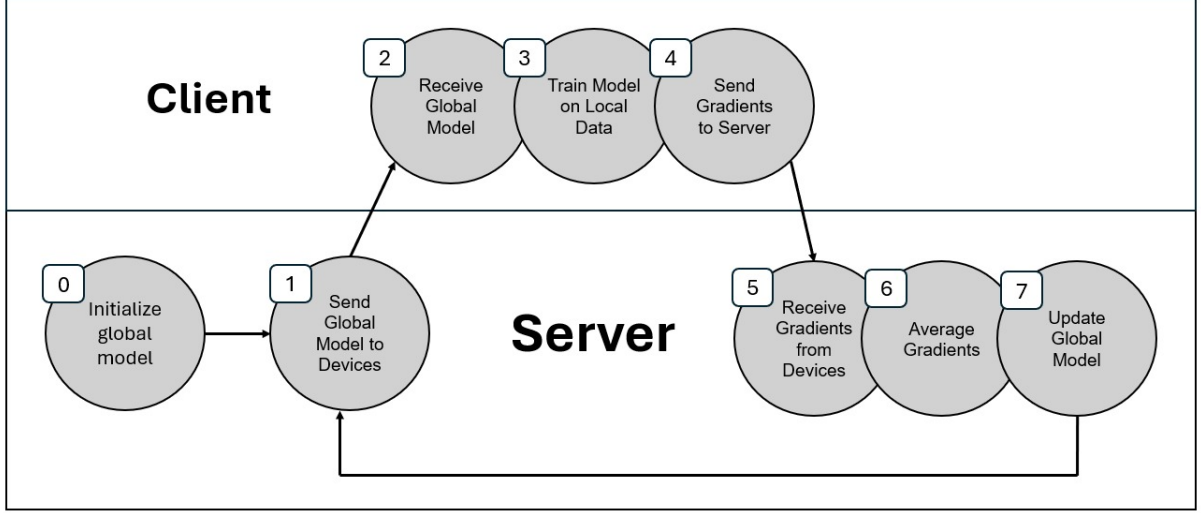


Figure 4.7: Federated Learning

Chapter 5

Evaluation

This chapter evaluates the DNN-FL model we described in the previous section using the ECU-IoHT dataset [8] and trained groups of five, ten, and fifteen clients. We also describe the scores we used to assess the performance of our model. The analysis highlights the impact of client number on model performance, providing insights into how data distribution effects learning results and identifying the optimal configuration for federated learning.

We evaluated our model using a laptop with the following specifications: an Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz with a turbo boost up to 2.30GHz, 8GB of installed RAM, and a 64-bit Windows 11 Pro operating system (version 23H2, OS build 22631.4602. In addition, we used Google Colab to run the model. The code for this project is publicly available on GitHub at <https://github.com/jenin144/Federated-Learning-Detection-System-in-IoHT-Devices>. This ensures the transparency and reproducibility of our outcomes.

5.1 Evaluation metrics

The performance of the proposed system is evaluated using a number of key scores. The result of a prediction for binary classification falls into one of four cases, where we consider attack as positive and normal as being negative. We usually represent the predictions by a confusion matrix from which we derive all metrics as shown in Figure 5.1

		Predicted	
		Negative	Positive
Actual	Negative	True Negative	False Positive
	Positive	False Negative	True Positive

Figure 5.1: confusion matrix

- TP true positive occurs when an attack is predicted and the actual result is attack.
- TN true negative occurs when the actual result and the predicted one are normal.
- FP false positive occurs When the actual result is normal while the prediction is attack.

- FN false negative occurs when the prediction is normal while the actual result is an attack.

Based on the definitions of True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) explained above, we can derive important metrics for evaluating the model's performance, such as Accuracy (Acc), Precision (Pre), Recall (Rec) and loss function. The formulas for these metrics are given below:

Accuracy:

The accuracy is the The percentage of correct predictions the model made on all samples, including normal and attacks. it gives an overall assessment of the model's performance, it may not be accurate for datasets that are unbalanced. In order to solve this, we used oversampling technique to guarantee class balance in our dataset. It is calculated using the following equation:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (5.1)$$

Precision:

The precision metric calculates the percentage of correctly identified positive examples out of all examples predicted as positive. It assesses the model's ability to avoid false positives, measuring how many of the predicted positive cases are actually correct. It is computed as follows:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (5.2)$$

Recall:

The recall calculates the percentage of the actual positive examples that are correctly identified by the model. It measures how well the model can detect all positive cases that are relevant while taking false negatives (instances where positives are missed) into account. This metric is very important in cases where failing to identify positive cases which is known as false negatives is costly. It is computed as follows:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (5.3)$$

Loss:

During model training, the loss function measures the difference between predicted values and actual targets. By reducing this difference, it directs optimization and guarantees improved predictions over time. The Cross-Entropy Loss is calculated using the following equation:

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i) \quad (5.4)$$

Where C represents the total number of classes, y_i is the true label for class i , and $\hat{y}_i$ represent the predicted probability for class i .

5.2 DNN-FL Model Results

In order to evaluate the model using federated learning, we compared the results in 3 different scenarios depending on the number of clients in each scenario. The dataset was first divided among 15 clients, then 10 clients, and finally 5 clients, ensuring that each client received an approximately equal share of the data. The process of this splitting is deterministic, without any randomness, Since the sharing is based only on the number of the rows in the dataset.

For each scenario, Key performance metrics were used to evaluate the ability of our to learn across multiple clients, including accuracy, precision, recall, training/testing time and loss. The time required to evaluate the performance of the model on the validation dataset after training is referred to as testing time. Furthermore, the overall time represents the total amount of time spent on the federated learning process, including all rounds, training, testing, and communication overhead. The obtained results on ECU-IoHT dataset [8] are summarized in Table 5.1 below.

Table 5.1: DNN-FL model results

No.of clients	Acc%	Pre%	Rec%	Train-Time (sec)	Test-Time (sec)	Overall Time (sec)	Loss
5	99.9	99.9	99.9	0.00011	0.00004	131	0.002
10	99.9	99.9	99.9	0.00006	0.00002	200	0.017
15	99.7	99.6	99.3	0.00009	0.00004	267	0.032

5.2.1 Summary of Table Results

The table highlights the performance metrics of the DNN-FL model for different client configurations (5, 10, and 15 clients) using the ECU-IoHT dataset [8]. The key metrics evaluated include accuracy (Acc%), precision (Pre%), recall (Rec%), training time (Train-Time), testing time (Test-Time), overall time, and loss.

Accuracy: With 5 and 10 clients, the model’s achieved high accuracy of 99.9%, but for fifteen clients, it slightly dropped to 99.7%. This indicates that there is a trade-off between accuracy and client count, with accuracy decreasing as client count increases.

Precision: With five clients, precision is at its highest (99.9%), and for ten and fifteen clients, it slightly decreases to 99.6%. This highlight that the model accurately detects positive cases (attacks) in all configurations with high effectiveness.

Recall: The ability of the model to detect all positive examples (attacks) decreases slightly as the number of clients increases, as evidenced by the highest recall of 99.9% for 5 and 10 clients, which slightly decreased to 99.3% for 15 clients.

training time The training is low across all scenarios. The training time is lowest for the 10-client configuration at 0.00006 seconds, compared to 0.00009 seconds for 15 clients and 0.00011 seconds for 5 clients. The training is highest for 5 clients (0.00011 seconds) due to larger data subsets handled by each client.

Testing time The testing time is the lowest for the 10-client configuration, recorded at 0.00002 seconds, compared to 0.00004 seconds for both 5 and 15 clients.

The **overall runtime** increases with the number of clients, from 131 seconds (5 clients) to 267 seconds (15 clients) This reflects the additional communication and aggregation overhead introduced as more clients participate in the federated learning process.

This could indicate that the model achieves a more balanced state during evaluation when 10 clients are used, possibly due to better data distribution and aggregation efficiency at this scale.

Loss: The lowest loss was observed with 5 clients (0.002), followed by 10 clients (0.017). The loss increased significantly to 0.032 for 15 clients, demonstrating that the ability of the model to generalize could decrease as the number of clients increase.

Chapter 6

Conclusion and Future Work

This chapter provides a comprehensive analysis of the model's performance in identifying cyberattacks on IoHT devices in multi-client environments, summarizing the final outcomes achieved by the proposed model. In addition, the challenges encountered during the implementation are highlighted, with a future work plan to improve performance, solve problems and expand the scope of application in the next semester.

6.1 Conclusion

The existence of policies and regulations that prevent sharing sensitive medical data, has made the use of machine learning systems to detect and mitigate cyber-attacks in IoHT devices a major challenge. Here comes Federated Learning with Deep Learning as a fundamental solution to enhance the security and privacy of this sensitive data. This is done by stopping the share of data in a central location, as the data is trained separately on each device without the need to send the data to the central server. In this work we present DNN based FL model using the ECU-IoHT dataset[8] to apply binary classification to distinguish between "attack" and "normal" cases. The experimental results showed that:

For 15 Clients:The training time for 15 clients is efficient at 0.00009 seconds due to smaller data subset for each client, but the overall time increases to 267 seconds due to higher communication overhead needed between the larger number of clients, also performance metrics decreased slightly, with accuracy dropping to 99.7%, precision to 99.6%, recall to 99.3%, and loss increases to 0.032.

For 10 Clients:This configuration shows excellent performance with a well-distributed workload among clients. This distribution achieves the most efficient testing time at 0.00002 seconds, indicating an optimal balance between client distribution and model synchronization. The accuracy is high at 99.9%, also precision and recall both at 99.9%. The overall time is 200 seconds, and the loss is 0.017.

For 5 Clients:The configuration with 5 clients achieves the best performance metrics, with accuracy, precision, and recall reaching 99.9%, and with lowest loss of 0.002, and shortest overall time of 131 seconds. However, it experiences slightly longer training and testing times compared to 10 clients due to larger data subsets assigned to each client.

These results highlight a trade-off between the number of clients and time efficiency for

training. As the number of clients increases, the overall time and loss increase due to increased communication and aggregation overhead, illustrating that the optimal configuration depends on whether training time, or model performance is the main concern. Also these results confirm the effectiveness of the proposed system in detecting intrusions and protecting health devices, while protecting it from leakage. Moreover, this model offers an innovative, scalable solution to a variety of medical devices in different environments. Nevertheless, there is still need to solve the Federated Learning challenges.

As mentioned earlier, Federated Learning faces several challenges:

Direct Data Leakage: Data leakage occurs through the possibility of reverse engineering the trained model or updates received, allowing the extraction of patterns or part of the data that was trained on if the model is not sufficiently protected.

Requires Expensive Computational Resources at the Clients' Side: Large computational resources, such as graphics processing units (GPUs) or advanced processors, on client devices are necessary for training the model, but these may not always be available, particularly in organizations with limited resources, like hospitals.

To solve these challenges, we propose the following solutions:

6.2 Future work

1. In order to solve the Problem of limited resources: we will train the models on cloud rather than directly on client devices. Each device will be assigned a private cloud to train its data separately, which maintains the independence of each device. To enhance the security of this approach, the data will be encrypted using homomorphic encryption technique before being sent to the cloud, we will sent also the encrypted gradients to the server to reduce the risk of data leakage.
2. To solve the Reverse Engineering Problem: We will encrypt all data and updates using Homomorphic Encryption techniques.
3. To improve Computational Efficiency: We will develop the system to support multi-classification instead of binary classification. Since training will be performed on the cloud, the available resources will be more efficient, increasing its ability to handle multiple types of attacks.
4. The model was designed with assumption that the data was homogeneous, but this assumption is inaccurate. Real data is often heterogeneous from one device to another. We will address this by designing a more flexible model that takes into account the diversity of the data.
5. In this work, we tested a federated learning model on a laptop, but it doesn't fully replicate real-world deployment conditions on IoHT devices, which typically have low resources such as limited computational power, memory, and network capabilities. Future work will focus on testing the model directly on IoHT devices to evaluate its performance under realistic conditions, including resource limitations and ability to detect anomalies or attacks. This will help identify potential optimizations or adaptations needed for system efficiency and practicality in real healthcare environments.

6.2.1 Future Plan

The following table details the plan we will follow to complete our project and solve the problems we faced, along with the expected time period for each task.

Table 6.1: Detailed Future plan.

Task Number	Description	Estimated Time (Weeks)
1	Cloud-based model training with encrypted data	6
2	Send encrypted updates to the server	3
3	Support multi-classification	2
4	Dealing with heterogeneous data	2
5	Evaluate the model on IoHT device	4

Bibliography

- [1] S. A. Algethami and S. S. Alshamrani, “A deep learning-based framework for strengthening cybersecurity in internet of health things (ioht) environments,” *Applied Sciences*, vol. 14, no. 11, p. 4729, 2024.
- [2] Statista Research Department, “Internet of Things (IoT) connected devices installed base worldwide from 2015 to 2025 [Online].” Available: <https://www.statista.com/statistics/471264/iot-number-of-connected-devices-worldwide>, 2016.
- [3] Statista Research Department, “Prognosis of worldwide spending on the Internet of Things (IoT) from 2018 to 2024 [Online].” Available: <https://www.statista.com/statistics/668996/worldwide-expenditures-for-the-internet-of-things>, 2016.
- [4] F. Mosaiyebzadeh, S. Pouriyeh, R. M. Parizi, M. Han, and D. M. Batista, “Intrusion detection system for ioht devices using federated learning,” in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pp. 1–6, IEEE, 2023.
- [5] E. I. Essa *et al.*, “Enhanced internet of things intrusion detection model using three-way selected random forest (3-wsrf),” *Nanotechnology Perceptions*, pp. 515–536, 2024.
- [6] J. Pope, T. Spyridopoulos, V. Kumar, F. Raimondo, S. Gunner, G. Oikonomou, T. Pasquier, R. McConville, P. Carnelli, A. Sanchez-Mompo, *et al.*, “Intrusion detection at the iot edge using federated learning,” in *Security and Privacy in Smart Environments*, pp. 98–119, Springer, 2024.
- [7] K. P. Vijayakumar, K. Pradeep, A. Balasundaram, and M. R. Prusty, “Enhanced cyber attack detection process for internet of health things (ioht) devices using deep neural network,” *Processes*, vol. 11, no. 4, p. 1072, 2023.
- [8] M. Ahmed, S. Byreddy, A. Nutakki, L. F. Sikos, and P. Haskell-Dowland, “Ecu-ioht: A dataset for analyzing cyberattacks in internet of health things,” *Ad Hoc Networks*, vol. 122, p. 102621, 2021.
- [9] L. Yang, M. Tian, D. Xin, Q. Cheng, and J. Zheng, “Ai-driven anonymization: Protecting personal data privacy while leveraging machine learning,” *arXiv preprint arXiv:2402.17191*, 2024.
- [10] “Lehigh valley health network data breach settlement.” <https://lvhndatabreachsettlement.com/#:~:text=The%20lawsuit%20alleges%20that%20Defendant,or%20about%20February%2022%2C%202023>. Accessed: 8 November 2024.

- [11] “Lehigh valley health network blackcat settlement.” <https://www.hipaajournal.com/lehigh-valley-health-network-blackcat-settlement/>. Accessed: 8 November 2024.
- [12] A. Ghourabi, “A security model based on lightgbm and transformer to protect health-care systems from cyberattacks,” *IEEE Access*, vol. 10, pp. 48890–48903, 2022.
- [13] UN News Global perspective Human stories, “Cyberattacks on healthcare: A global threat that can’t be ignored.” Available: <https://news.un.org/en/story/2024/11/1156751>, 8 November 2024.
- [14] E. Saleh, A. Alsa’deh, A. Kayed, and C. Meinel, “Processing over encrypted data: between theory and practice,” *ACM SIGMOD Record*, vol. 45, no. 3, pp. 5–16, 2016.
- [15] J. Ma, S.-A. Naas, S. Sigg, and X. Lyu, “Privacy-preserving federated learning based on multi-key homomorphic encryption,” *International Journal of Intelligent Systems*, vol. 37, no. 9, pp. 5880–5901, 2022.
- [16] J. Ranjith, K. Mahantesh, and C. Abhilash, “Lw-pwecc: Cryptographic framework of attack detection and secure data transmission in iot,” *Journal of Robotics and Control (JRC)*, vol. 5, no. 1, pp. 228–238, 2024.
- [17] G. Lazrek, K. Chetioui, Y. Balboul, S. Mazer, *et al.*, “An rfe/ridge-ml/dl based anomaly intrusion detection approach for securing iomt system,” *Results in Engineering*, vol. 23, p. 102659, 2024.
- [18] U. Zukaib, X. Cui, C. Zheng, M. Hassan, and Z. Shen, “Meta-ids: Meta-learning based smart intrusion detection system for internet of medical things (iomt) network,” *IEEE Internet of Things Journal*, 2024.
- [19] P. Lanka, K. Gupta, and C. Varol, “Intelligent threat detection—ai-driven analysis of honeypot data to counter cyber threats,” *Electronics*, vol. 13, no. 13, p. 2465, 2024.
- [20] V. Rey, P. M. S. Sánchez, A. H. Celdrán, and G. Bovet, “Federated learning for malware detection in iot devices,” *Computer Networks*, vol. 204, p. 108693, 2022.